Hierarchical Graph Structural Aggregation for Cross-Architecture IoT Malware Detection under Confound-Controlled Evaluation

Xuan Thanh Dinh[1], Anh Hoang[2⋆[0000−0003−2754−3710]], and Thu Ha Tran[3]

[1] *Faculty of Computer Science and Engineering*, Ho Chi Minh City University of Technology (HCMUT), Ho Chi Minh City, Vietnam
thanh.dinh2453165@hcmut.edu.vn

[2] University of Economics Ho Chi Minh City (UEH), Ho Chi Minh City, Vietnam
Anhh@ueh.edu.vn

[3] *Faculty of Business Information Technology*, University of Economics Ho Chi Minh City (UEH), Ho Chi Minh City, Vietnam
hatran.31241024918@st.ueh.edu.vn

Abstract. The rapid proliferation of Internet of Things (IoT) devices across heterogeneous processor architectures has significantly expanded the cyber-attack surface. Dynamic emulation-based detectors, while effective, incur prohibitive computational overhead and remain vulnerable to environment-aware evasion, making them impractical for resource-constrained edge gateways. Existing static approaches construct monolithic flat Control Flow Graphs (CFGs) that suffer from information dilution at scale, or rely on inter-procedural call graphs that are frequently disrupted by compiler obfuscation.

To address these limitations, we propose H-GNN, a hierarchical graph neural network framework for cross-architecture IoT malware detection. H-GNN lifts multi-architecture binaries (ARM, MIPS, PowerPC, x86) into a unified VEX Intermediate Representation mapped to a compact vocabulary of VEX statement tags. Binary structure is modeled at two levels: bounded intra-procedural CFGs (Kmax = 50) encode function-level semantics via GATv2 and maximum pooling, while the global program topology leverages the true inter-procedural call graph to aggregate function embeddings via GraphSAGE and global max-pooling for final classification. Evaluated on a confound-controlled, cross-architecture dataset of 3,860binaries under a family-disjoint split, H-GNN achieves an F1-score of $97.99\% \pm 0.44\%$ and accuracy of 97.69%. On the same dataset a metadata-only classifier drops to 0.40 accuracy once file size is excluded, well below the 0.99 it reaches on a naive split, indicating that the model's decisions derive from program structure rather than build provenance.

Keywords: Hierarchical Graph Neural Networks · Cross-Architecture Detection · IoT Malware Analysis · Static Binary Analysis · Intermediate Representation · Inductive Graph Learning · VEX IR Semantic Mapping

⋆ Corresponding author.

1 Introduction

The Internet of Things (IoT) ecosystem underpins modern smart infrastructures and edge computing, yet its rapid proliferation across heterogeneous processor architectures (ARM, MIPS, x86, PowerPC) has substantially expanded the cyber-attack surface [9]. Cross-architecture malware families such as *Mirai*, *Gafgyt*, and *Tsunami* exploit this diversity by compiling identical source code into radically distinct binary layouts per Instruction Set Architecture (ISA), making unified detection inherently difficult [2].

Dynamic emulation-based detectors analyze runtime system call trajectories and execution flows within sandboxed environments [4]. While effective against advanced obfuscation, they incur prohibitive computational overhead and remain vulnerable to environment-aware evasion, rendering them impractical for resource-constrained edge gateways. Static approaches avoid execution entirely by inspecting binary structure, but mainstream frameworks construct monolithic flat Control Flow Graphs (CFGs) that suffer from *information dilution* at scale: during message passing, sparse malicious footprints are overwhelmed by the vast majority of benign code nodes, a known limitation of flat graph representations at scale. An alternative — tracking global function interactions via traditional Inter-procedural Call Graphs (ICGs) — also struggles across ISAs, as symbol-table-based call targets are frequently stripped or obfuscated by cross-compilers [6], leading to topological fragmentation.

To address these limitations, we leverage multi-architecture binary artifacts sourced from public IoT malware corpora to propose H-GNN, a hierarchical graph neural network framework for static cross-architecture IoT malware detection. H-GNN decouples binary analysis into two levels. At the intra-procedural level (Level 1), raw binaries are lifted into architecture-independent VEX Intermediate Representation (IR) and mapped to a compact VEX statement-tag vocabulary; bounded intra-procedural CFGs ($K_{\max} = 50$) are constructed per function to isolate core semantics and suppress library noise via maximum pooling. At the inter-procedural level (Level 2), instead of relying on fragile symbol tables, H-GNN reconstructs the explicit structural call topology directly from the VEX instruction text segment to form a resilient Inter-procedural Call Graph, aggregated via GraphSAGE and global max-pooling for final classification. This design prevents node explosion and mitigates topological fragmentation without relying on obfuscation-prone external symbol resolutions.

We study two research questions: RQ1, detection performance under confound-controlled, family-disjoint evaluation; and RQ3, the contribution of each structural component. Cross-ISA generalisation and on-device efficiency are left to future work.

The main contributions of this work are as follows:

- Architecture-agnostic binary lifting: A cross-architecture preprocessing pipeline that normalizes ARM, MIPS, PowerPC, and x86 instructions into a unified VEX IR space mapped to a compact statement-tag vocabulary, establishing a platform-independent feature space for heterogeneous IoT binaries.
- Hierarchical Graph Neural Network (H-GNN): A dual-stage graph learning architecture coupling bounded intra-procedural CFG encoding (GATv2) with an explicit inter-procedural call graph topology (GraphSAGE + global max-pooling), preventing node explosion and resolving information dilution in large binary graphs.

– Confound-controlled evaluation: Comprehensive evaluation on a confound-controlled, cross-architecture dataset of 3,860 binaries under a family-disjoint split. H-GNN achieves an F1-score of $97.99\% \pm 0.44\%$ and accuracy of 97.69%, with decisions driven by program structure rather than build provenance.

2 Related Work

2.1 Cross-Architecture IoT Malware Detection

The diversity of IoT hardware has driven the proliferation of multi-architecture malware variants targeting ARM, MIPS, and x86 devices [2]. Defense approaches fall into two broad categories.

Dynamic analysis executes binaries inside sandboxes or emulators to monitor system call sequences and live control flow [4]. Although dynamic traces are robust against syntactic obfuscation, this paradigm introduces severe processing latency, high CPU/RAM overhead, and susceptibility to environment-aware evasion — all of which make it unsustainable on resource-constrained edge gateways [9].

Static analysis inspects binary structure without execution. Some pipelines lift raw binaries into architecture-agnostic VEX Intermediate Representation (IR) via *angr* before downstream analysis. This should be distinguished from Xunpack, a cross-architecture unpacking tool [6], and Gemini, which represents functions as Attributed Control Flow Graphs (ACFGs) extracted using IDA and embeds them with Structure2Vec for binary similarity [12]. Chaganti *et al.* [3] demonstrate cross-architecture IoT malware classification via deep learning on lifted instruction sequences.

2.2 Graph-Based Detection and Research Gaps

Graph Neural Networks (GNNs) model non-Euclidean program structure by applying neighbourhood message-passing over graphs such as Control Flow Graphs (CFGs) to aggregate node features. Approaches such as Gemini [12], DeepBinDiff [5], and VulHawk [8] demonstrate strong binary analysis performance on graph representations. However, two critical gaps persist in existing graph-based IoT malware detectors. Separately, Abusnaina et al. study adversarial attacks against graph-based IoT malware detection systems, highlighting their exposure to adversarial manipulation [1].

First, some frameworks construct monolithic, program-wide flat CFGs by merging all basic blocks into a single graph. As binary size grows, these flat topologies suffer from *information dilution*: sparse malicious behavioural signals are iteratively overwhelmed by the dominant benign background during message propagation, causing classification accuracy to degrade. Second, approaches that derive inter-procedural call graphs from external symbol tables may become fragile when binaries are stripped or obfuscated, fragmenting the global graph topology.

H-GNN resolves both gaps by decoupling the two levels of program structure. Bounded intra-procedural CFGs isolate function-level semantics and suppress library noise via local maximum pooling; meanwhile, the global level reconstructs an explicit Inter-procedural Call Graph by parsing control-flow transfer primitives directly within

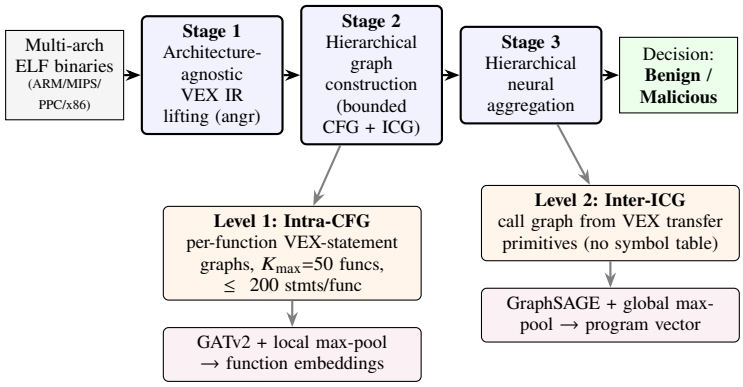


Fig. 1. Overview of the proposed H-GNN framework. Heterogeneous multi-architecture ELF binaries are lifted into architecture-agnostic VEX IR, decomposed into a two-level hierarchical graph (bounded intra-procedural CFGs and an inter-procedural call graph reconstructed from control-flow primitives), and encoded bottom-up with GATv2 and GraphSAGE for binary malware detection.

the lifted VEX IR space, thereby bypassing symbol-table dependencies and preserving macroscopic program state. This design propagates malicious contexts via inductive GraphSAGE convolutions and global max-pooling readout, enabling accurate static inference across heterogeneous ISAs without severe information dilution or topological fragmentation.

Prior work has also explored hierarchical graph representations combining a program-level Function Call Graph (FCG) with function-level Control Flow Graphs (CFGs). MalGraph targets Windows PE binaries and relies on IDA and symbol information [7], while Wang et al. study hierarchical graph-based malware detection [11]. In contrast, H-GNN targets multi-architecture IoT ELF binaries, constructs the call graph directly from VEX IR without requiring a symbol table, and evaluates inference under a confound-controlled, family-disjoint protocol.

3 Proposed Methodology

3.1 System Overview

The proposed H-GNN framework is architected as a modular, high-throughput static analysis pipeline designed to process heterogeneous IoT binaries and generate an accurate classification decision (Malicious or Benign). As illustrated in Fig. 1, the pipeline bypasses the vulnerabilities of traditional static methods by decoupling program representation into three distinct, interconnected stages:

1. Architecture-Agnostic Preprocessing Layer: Normalizes multi-architecture binaries (ARM, MIPS, x86, PowerPC) by lifting machine instructions into a unified Intermediate Representation (VEX IR) via the *angr* framework.

2. Hierarchical Graph Construction Layer: Extracts localized Control Flow Graphs (CFGs) for individual functions under a bounded constraint to eliminate structural noise (Level 1), then constructs the true Inter-procedural Call Graph to explicitly represent global functional dependencies without fragile flat sequence conversion (Level 2).
3. Hierarchical Neural Aggregation Engine: Executes bottom-up representation learning where a local GNN function encoder processes function-level CFGs via maximum pooling, which are subsequently aggregated and refined by a global GraphSAGE processing layer and a global max-pooling readout module for final inference.

3.2 Threat Model and Scope

We target static detection of cross-architecture IoT malware without executing samples. The adversary distributes ELF binaries cross-compiled for heterogeneous ISAs (ARM, MIPS, PowerPC, x86), which may be stripped, statically linked, and packed (e.g., UPX), and may undergo lightweight structural perturbation such as function insertion, reordering, or dead-code injection. The defender inspects binary structure only. Out of scope are samples that the unpacking stage cannot recover, heavily obfuscated or self-modifying binaries that prevent reliable lifting, and active adversarial evasion against the model, which we identify as future work.

3.3 Architecture-Agnostic Preprocessing and Feature Initialization

To achieve cross-architecture compatibility, the preprocessing layer strips hardware-dependent properties prior to graph building, transforming raw machine code into platform-independent VEX IR statements [10].

- Intermediate Representation Decomposition: VEX IR abstracts away architecture-specific register layouts and instruction sets, converting complex CISC/RISC operations into side-effect-free, standardized primitives. Instruction semantics are condensed into core operational behaviors such as register reads (`GET`), memory loads (`LDle`), stores (`STle`), and arithmetic operations (`Add32`).
- Semantic Token Mapping: Each lifted function is represented at the VEX statement level. Every VEX statement is assigned one of a compact vocabulary of statement tags (e.g., `WrTmp`, `Put`, `Store`, `Exit`, `LoadG`), and each node is embedded through a trainable embedding layer $\mathbf{x} \in \mathbb{R}^d$ rather than a fixed one-hot vector. This yields a low-dimensional, architecture-independent feature space learned end-to-end with the classifier.

3.4 Hierarchical Graph Layout Construction

To preserve localized control logic while scaling the graph representation, H-GNN models binary software using a decoupled, dual-level hierarchical structure.

Level 1: Intra-procedural Control Flow Graph (CFG) For each isolated function f_i extracted from the binary, a local Control Flow Graph is formulated as:

$$G_{\text{local}}^{(i)} = (V_{\text{local}}^{(i)}, E_{\text{local}}^{(i)}), \tag{1}$$

where $V_{\text{local}}^{(i)}$ is the set of VEX statement nodes of function f_i (each embedded from its statement tag), and $E_{\text{local}}^{(i)}$ contains both sequential control-flow edges (within and across basic blocks) and data-dependency edges induced by VEX temporaries.

To eliminate structural noise from redundant library routines and prevent memory explosion, we introduce a strict structural constraint $K_{\max} = 50$. The local graph extraction space is bounded such that:

$$i \in \{1, 2, \ldots, N\} \quad \text{where} \quad N = \min(\text{Num_Functions}, K_{\max}). \tag{2}$$

For reproducibility, when a binary exceeds $K_{\max}$ the first $K_{\max}$ functions in ascending order of virtual entry address are selected. This bound limits graph size and reduces the information dilution introduced by large numbers of helper routines. We acknowledge that an adversary performing function insertion or reordering (Section 3) could shift malicious routines outside this address-ordered window; assessing robustness to such perturbation is left to future work.

Level 2: Inter-procedural Call Graph (ICG) To capture the macroscopic context and control transitions across separate subroutines without flattening structural dependencies, H-GNN models the global program level using the explicit execution topology of the Inter-procedural Call Graph. The global program architecture is formulated as a directed graph layout:

$$G_{\text{global}} = (V_{\text{global}}, E_{\text{global}}), \tag{3}$$

where each functional vertex $v_i \in V_{\text{global}}$ corresponds to a dense embedding vector $\mathbf{h}_f^{(i)}$ representing the localized execution state of the i-th bounded local function CFG ($i \leq N$), and each directed edge $e_{ij} \in E_{\text{global}}$ signifies an explicit inter-procedural function call interaction extracted sequentially from the binary text segment.

3.5 Hierarchical Neural Aggregation Engine

The network inference engine executes reasoning over the hierarchical topology via a dual-stage bottom-up aggregation process.

Stage 1: Intra-Procedural Semantic Encoding Each bounded local CFG ($G_{\text{local}}^{(i)}$) is processed through a localized Graph Neural Network using the Graph Attention Network v2 (GATv2) architecture. The dynamic attention coefficient α_{jk} between two connected nodes j and k within the local graph is evaluated as:

$$\alpha_{jk} = \frac{\exp\left(\mathbf{a}^T \sigma\left(\mathbf{W}[\mathbf{h}_j \| \mathbf{h}_k]\right)\right)}{\sum_{m \in \mathcal{N}(j)} \exp\left(\mathbf{a}^T \sigma\left(\mathbf{W}[\mathbf{h}_j \| \mathbf{h}_m]\right)\right)}, \tag{4}$$

where $\mathcal{N}(j)$ denotes the spatial neighborhood of node j, $\mathbf{W}$ is a shared weight transformation matrix, $\|$ signifies concatenation, and $\mathbf{a}$ represents the dynamic attention weight vector.

The updated representation of node i aggregates its neighbours weighted by these coefficients:

$$\mathbf{h}'_i = \sigma\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}\, \mathbf{W}\, \mathbf{h}_j\right), \tag{5}$$

The intra-procedural encoder stacks two GATv2 layers: the first uses multi-head attention with concatenation, and the second uses a single attention head.

Following iterative message-passing, the updated representations are compressed into a singular function-level embedding vector $\mathbf{h}_f^{(i)} \in \mathbb{R}^D$ via an anisotropic local maximum pooling readout phase:

$$\mathbf{h}_f^{(i)} = \mathrm{ReLU}\left(\mathbf{W}_f \cdot \left[\max_{v \in V_{\text{local}}^{(i)}} \mathbf{h}_v\right]\right), \tag{6}$$

where $\mathbf{W}_f$ denotes the functional transformation matrix. Max-pooling preserves sparse, localized signals within a function that averaging would dilute.

Stage 2: Inter-Procedural Propagation and Readout Phase The N function embeddings are deployed onto the vertices of the Inter-procedural Call Graph to initialize the program state matrix representing the global executable distribution:

$$\mathbf{X}_{\text{program}} = \left[\mathbf{h}_f^{(1)} \| \mathbf{h}_f^{(2)} \| \ldots \| \mathbf{h}_f^{(N)}\right]^T \in \mathbb{R}^{N \times D}. \tag{7}$$

To dynamically diffuse malware characteristics across function boundaries and track contextual dependencies, an inter-procedural convolution layer ($\mathrm{GNN}_{\text{global}}$, instantiated via inductive GraphSAGE convolutions) is executed over the true call graph adjacency structure $\mathbf{A}_{\text{call}}$ extracted from the binary text segment:

$$\mathbf{X}_{\text{refined}} = \mathrm{ReLU}\left(\mathrm{GNN}_{\text{global}}(\mathbf{X}_{\text{program}}, \mathbf{A}_{\text{call}})\right). \tag{8}$$

The underlying inter-procedural message passing routine at an arbitrary functional network layer l is formalized as follows:

$$\mathbf{h}_{v_i}^{(l)} = \mathrm{ReLU}\left(\mathbf{W}^{(l)} \cdot \left[\mathbf{h}_{v_i}^{(l-1)} \,\|\, \mathrm{MEAN}\left(\{\mathbf{h}_{v_j}^{(l-1)}, \forall v_j \in \mathcal{N}(v_i)\}\right)\right]\right), \tag{9}$$

where $\mathcal{N}(v_i)$ signifies the set of immediate neighboring subroutines interacting with v_i within $\mathbf{A}_{\text{call}}$, and $\mathbf{W}^{(l)}$ denotes the trainable structural weight matrix.

To obtain a program-level representation, H-GNN applies global max-pooling across the refined call graph:

$$\mathbf{g}_{\text{program}} = \max_{i=1}^{N} \mathbf{X}_{\text{refined}}[i, :]. \tag{10}$$

This hierarchical pooling workflow pairs local max-pooling within function blocks with global max-pooling over the inter-procedural call graph, selecting the most salient

functional responses at both levels; an empirical comparison (Section 4.3) shows this outperforms global mean-pooling on our dataset. The unified representation vector $\mathbf{g}_{\text{program}}$ is then projected onto the output classification space, where the final inference probability vector $\hat{\mathbf{y}}$ is derived via a LogSoftmax activation:

$$\hat{\mathbf{y}} = \text{LogSoftmax}\left(\mathbf{W}_c \mathbf{g}_{\text{program}} + \mathbf{b}_c\right), \tag{11}$$

where $\mathbf{W}_c$ and $\mathbf{b}_c$ denote the classifier weight matrix and bias vector, mapping the structural verdict to either the benign or the malicious class.

4 Experimental Results and Analysis

We evaluate H-GNN on two research questions our current setup answers directly: **RQ1** (detection performance on a confound-controlled, cross-architecture dataset) and **RQ3** (the contribution of each structural component). Cross-ISA generalisation (RQ2) and on-device efficiency (RQ4) are discussed as future work in Section 6.

4.1 Dataset and Experimental Setup

A central methodological concern in cross-architecture IoT malware detection is that benign and malicious samples often differ in build provenance—architecture, linking, stripping, packing, and compiler toolchain—rather than in behaviour, letting a classifier reach high accuracy by learning artefacts instead of maliciousness. To control for this, the benign class is compiled by the authors from open-source software using period and modern cross-compilation toolchains, so that its architecture, ABI, linking, stripping, and packing distributions match the malware class. We quantify residual leakage with a metadata-only probe (Section 5): on a naive benign/malware split it reaches roughly ∼0.99 accuracy, whereas on our matched dataset it drops to 0.40 once file size is excluded, indicating that low-level build artefacts no longer separate the classes.

The dataset contains 3,860 ELF binaries over six instruction-set targets (ARM OABI/EABI, MIPS big/little endian, PowerPC, i386). These six targets span four ISA families (ARM, MIPS, PowerPC, x86). Malware is drawn from public IoT corpora, deduplicated by SHA-256, restricted to architectures supported by our lifter, and packed malware samples are UPX-unpacked prior to analysis, while a matched fraction of the benign set is UPX-packed to mirror the malware packing ratio. Benign samples span 281 distinct open-source programs. We use a *family-/program-disjoint* split so that programs and malware families at test time are unseen during training: 2,691 training, 526 validation, and 554 test graphs.

Each binary is lifted to VEX IR and decomposed into at most $K_{\max} = 50$ bounded intra-procedural graphs, each capped at 200 VEX statements. Models train for 30 epochs (Adam, lr 2×10^{-4}, weight decay 10^{-4}, batch size 8, NLL loss); the best-validation-F1 epoch is kept for testing. Results are mean ± std over five seeds unless noted.

4.2 RQ1: Detection Performance

Table 1 reports test-set performance. H-GNN attains F1 97.99 ± 0.44% with PR-AUC 99.55% and a false-positive rate of 1.29% at 95% true-positive rate. These numbers are

Table 1. Detection performance of H-GNN on the family-disjoint test set (mean ± std, five seeds).

Metric	Value (%)
F1-score	97.99 ± 0.44
Accuracy	97.69 ± 0.51
Precision	96.99 ± 1.06
Recall	99.02 ± 0.63
MCC	95.31 ± 1.04
PR-AUC	99.55 ± 0.34
FPR @ 95% TPR	1.29 ± 1.56

Table 2. Component ablation (test F1). Removing the hierarchical decomposition causes the largest degradation. The no-hierarchy row is the mean of two seeds.

Configuration	F1 (%)
H-GNN (full, global max-pool)	97.99 ± 0.44
global max-pool → mean-pool	95.77 ± 0.67
w/o hierarchical decomposition	89.62

obtained where a metadata-only classifier drops well below the naive-split baseline once file size is removed, so the model's decisions derive from program structure rather than build provenance. The headline F1 is therefore an honest measure of malicious structure under a strict, family-disjoint protocol.

Operational precision under class imbalance. Because malware is rare in deployment, we report precision under realistic base rates rather than only the balanced test set. As malware prevalence falls to 10%, 1%, and 0.1%, precision drops to 0.74, 0.22, and 0.03 respectively, reflecting the base-rate effect inherent to rare-event detection. We state this explicitly so that the required operating point for gateway deployment (e.g. pairing H-GNN with a lightweight pre-filter to raise effective prevalence) is clear rather than obscured by a balanced-set figure.

4.3 RQ3: Ablation Study

We retrain H-GNN with individual components removed under the same protocol (Table 2, Fig. 2).

Two findings emerge. First, collapsing the two-level hierarchy into a single flat graph reduces F1 from 97.99% to 89.62% (about 8 points), confirming that separating intra-procedural semantics from inter-procedural structure is the dominant contributor. Second, replacing global max-pooling with mean-pooling reduces F1 to 95.77%, indicating that selecting the most salient functional responses is more effective than averaging them here; we adopt global max-pooling in the final model.

5 Threats to Validity

Construct validity. A key risk in cross-architecture malware datasets is that benign and malicious samples differ in build provenance (architecture, linking, stripping, packing,

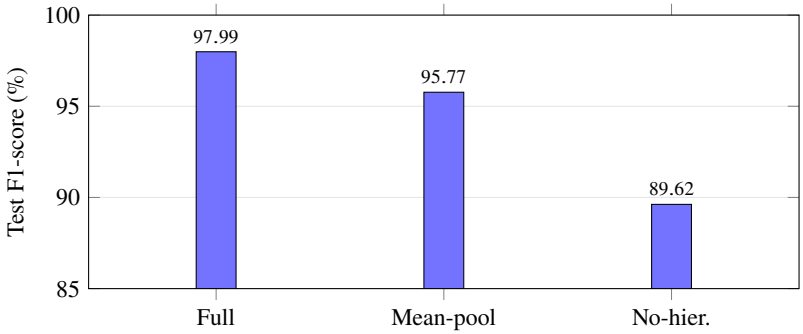


Fig. 2. Component ablation (test F1-score; y-axis truncated at 85 for readability). The full H-GNN with hierarchical decomposition and global max-pooling attains 97.99%; replacing global max-pooling with mean-pooling drops it to 95.77%, and removing the hierarchical decomposition drops it to 89.62% (mean of two seeds), confirming both design choices.

compiler) rather than in behaviour, allowing a classifier to exploit these artefacts. To control this, we construct a benign set whose architecture, ABI, linking, stripping, and packing distributions are matched to the malware set, and quantify residual leakage with a metadata-only probe. On a naive benign/malware split the probe reaches roughly ∼0.99 accuracy, whereas on our matched dataset it drops to 0.40 once file size is excluded, indicating that architecture, linking, stripping, and packing no longer separate the classes. Residual file-size differences are handled at the model level: H-GNN operates on CFG structure, and the $K_{\max}$ bound limits graph size.

Internal validity. To avoid train/test leakage from near-duplicate malware variants and shared benign programs, we deduplicate by SHA-256 and evaluate under a family-/program-disjoint split, where test programs and malware families are unseen during training.

External validity. The benign set is assembled from a finite set of open-source programs and two toolchain generations; it may not capture all legitimate IoT software. We report the number of distinct benign programs and include both period and modern toolchains to mitigate compiler-era bias.

Conclusion validity. We report mean and standard deviation across five random seeds on a fixed family-disjoint split to reduce sensitivity to a single partition.

6 Conclusion and Future Work

We introduced H-GNN, a hierarchical GNN framework for cross-architecture IoT malware detection that couples intra-functional VEX-IR CFG semantics with an explicit inter-procedural call graph, enabling architecture-agnostic threat identification across heterogeneous ISAs. Experimental evaluations on a confound-controlled, family-disjoint dataset confirm that H-GNN achieves an F1-score of 97.99%, with ablation studies confirming the necessity of each hierarchical component.

Moving forward, we envision several promising directions for future exploration:

- Limitations: H-GNN currently assumes that *angr* can successfully lift the target binary; heavily packed or self-modifying binaries may require an unpacking pre-stage before analysis.
- Scalability and Latency Optimization: We aim to integrate graph sparsification and sub-graph partitioning techniques to reduce computational overhead in high-throughput enterprise environments.
- Adversarial Graph Resilience: We will investigate defensive training against graph evasion tactics such as dead-code injection, function splitting, and control-flow obfuscation.
- Explainable AI (XAI) for Threat Intelligence: We intend to embed explainability frameworks (*e.g.*, GNNExplainer) to help analysts localize malicious sub-graphs and convert neural predictions into actionable cyber threat intelligence.
- Cross-ISA generalisation (leave-one-ISA-out) and on-device latency/energy measurement on gateway hardware.

Data and Code Availability

Our source code and dataset manifest — including the SHA-256 hash of every sample and the exact train/validation/test partition — are available in a public repository (URL provided upon acceptance). For safety, malware binaries are not redistributed; only their hashes are released so the evaluation set can be reconstructed from public malware repositories. The benign set is compiled by the authors from open-source software, and the build scripts are included.

References

1. Abusnaina, A., Khormali, A., Alasmary, H., Park, J., Anwar, A., Mohaisen, A.: Adversarial learning attacks on graph-based IoT malware detection systems. In: Proceedings of the IEEE International Conference on Distributed Computing Systems (ICDCS). pp. 1296–1305 (2019)
2. Alrawi, O., Lever, C., Valakuzhy, K., Court, R., Snow, K., Monrose, F., Antonakakis, M.: The circle of life: A large-scale study of the IoT malware lifecycle. In: Proceedings of the USENIX Security Symposium. pp. 3505–3522 (2021)
3. Chaganti, R., Ravi, V., Pham, T.D.: Deep learning based cross architecture internet of things malware detection and classification. Computers & Security **120**, 102779 (2022)
4. Cozzi, E., Graziano, M., Fratantonio, Y., Balzarotti, D.: Understanding Linux malware. In: IEEE Symposium on Security and Privacy (SP). pp. 161–175 (2018)
5. Duan, Y., Li, X., Wang, J., Yin, H.: DeepBinDiff: Learning program-wide code representations for binary diffing. In: Proceedings of the Network and Distributed System Security Symposium (NDSS) (2020)
6. Kawakoya, Y., Akabane, S., Iwamura, M., Okamoto, T.: Xunpack: Cross-architecture unpacking for Linux IoT malware. In: Proceedings of the International Symposium on Research in Attacks, Intrusions and Defenses (RAID). pp. 471–484 (2023)
7. Ling, X., Wu, L., Deng, W., Qu, Z., Zhang, J., Zhang, S., Ma, T., Wang, B., Wu, C., Ji, S.: MalGraph: Hierarchical graph neural networks for robust Windows malware detection. In: Proceedings of IEEE INFOCOM. pp. 1998–2007 (2022). https://doi.org/10.1109/INFOCOM48880.2022.9796786

8. Luo, Z., et al.: VulHawk: Cross-architecture vulnerability detection with entropy-based binary code search. In: Proceedings of the Network and Distributed System Security Symposium (NDSS) (2023)
9. Roman, R., Lopez, J., Mambo, M.: Mobile edge computing, fog et al.: A survey and analysis of security threats and challenges. Future Generation Computer Systems **78**, 680–698 (2018)
10. Shoshitaishvili, Y., Wang, R., Salls, C., Stephens, N., Polino, M., Dutcher, A., Grosen, J., Feng, S., Hauser, C., Kruegel, C., Vigna, G.: Sok: (state of) the art of war: Offensive techniques in binary analysis. In: IEEE Symposium on Security and Privacy (SP). pp. 138–157 (2016)
11. Wang, S., Zhao, Y., Liu, G., Su, B.: A hierarchical graph-based neural network for malware classification. In: Neural Information Processing (ICONIP 2021). Lecture Notes in Computer Science, vol. 13111, pp. 621–633 (2021). https://doi.org/10.1007/978-3-030-92273-3_51
12. Xu, X., Liu, C., Feng, Q., Yin, H., Song, L., Song, D.: Neural network-based graph embedding for cross-platform binary code similarity detection. In: Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS). pp. 363–376 (2017)